Smart College Registration

AWS project build and interview guide

A beginner-friendly explanation of the web app, AWS services, how the pieces connect, and how the design could be improved.

Project in one sentence I built a student registration web app where submissions are queued in Amazon SQS, processed by AWS Lambda, stored in DynamoDB, and optionally sent as SNS notifications. An Application Load Balancer (ALB) routes incoming web traffic to the application.

What this guide covers

- The request flow from the browser to the database.
- How to create the VPC, networking components, and AWS services in a sensible order.
- The purpose of each service, the key settings, testing, and common cost risks.
- A professional list of future improvements for a production-ready version.

Scope This guide describes the project architecture based on the available project files. Exact VPC IDs, subnet choices, and account settings may differ. Sample values are examples for recreating the project in your own AWS account.

1. Project overview

The project has two programs. The Flask application runs in a Docker container on ECS and handles the registration form. The Lambda function runs separately and processes queue messages. SQS separates the web request from database processing, so the app can respond after the message is accepted by the queue.

Part	What it does
Flask app	Shows the student form, validates required fields, creates a student ID, and sends JSON to SQS.
ECS and Docker	Run the Flask app as a containerized service.
SQS	Holds registration messages until Lambda reads them.
Lambda	Reads SQS messages and writes student records to DynamoDB.
DynamoDB	Stores student records with student_id as the partition key.
SNS (optional)	Sends a notification after processing if a topic and subscription are configured.
ALB and ECR	The ALB receives web traffic and routes it to ECS. ECR stores the Docker image.

The supplied Dockerfile copies app.py and the templates folder. It does not copy lambda_function.py. The Lambda source is deployed separately.

2. Architecture and data flow

Step	What happens
1	A browser opens the ALB address. The ALB forwards HTTP traffic to the Flask container on port 5000.
2	The student submits the form. Flask checks required fields and sends a JSON message to SQS.
3	Flask returns a confirmation when SQS accepts the message. This confirms queueing, not final database storage.
4	The Lambda SQS event source mapping polls the queue and invokes the Lambda handler with records.
5	Lambda parses each message and saves it to DynamoDB. It can then publish an SNS notification.
6	CloudWatch Logs help diagnose Lambda processing errors. DynamoDB confirms whether a record was saved.

Simple explanation "I used SQS between the form and the database so the web app could hand off each registration. Lambda processes queued messages and writes them to DynamoDB."

3. Network foundation VPC subnets and gateways

A VPC is a logically isolated network in AWS. Subnets divide that network across Availability Zones. Route tables decide where subnet traffic goes. For a production-style design, put the ALB in public subnets and ECS tasks in private subnets.

A. Create a custom VPC

1. Open VPC in the AWS Console, choose **Your VPCs**, then **Create VPC**.
2. Choose **VPC only**. Name it, for example, college-demo-vpc, and use an example IPv4 CIDR such as 10.0.0.0/16. The CIDR is the range of network addresses.
3. Create the VPC. Enable DNS resolution and DNS hostnames if the project configuration needs them.

B. Create subnets in two Availability Zones

For a small multi-AZ layout, create two public and two private subnets. Example CIDRs: 10.0.1.0/24 and 10.0.2.0/24 for public subnets; 10.0.11.0/24 and 10.0.12.0/24 for private subnets. Each range must be inside the VPC range and must not overlap.

1. In VPC, choose **Subnets**, then **Create subnet**. Select the VPC and an Availability Zone for each subnet.
2. Create each subnet with its own CIDR. Use clear names such as public-a, public-b, private-a, and private-b.
3. Enable automatic public IPv4 assignment for public subnets only when the chosen design needs it. ECS tasks do not need to be publicly reachable just because the ALB is public.

C. Attach an Internet Gateway

1. Choose **Internet gateways**, create an IGW such as college-demo-igw, then attach it to the VPC.
2. Create a public route table, associate both public subnets with it, and add a default route 0.0.0.0/0 targeting the Internet Gateway. Keep the local route AWS adds automatically.

A route makes a subnet public, but does not expose every resource by itself. Security groups and public IP settings also matter.

D. Private subnet outbound access

If private ECS tasks need outbound internet access, a common design places a NAT Gateway in a public subnet. Allocate an Elastic IP, create the NAT Gateway, wait until it is available, then add a default route from the private route table to the NAT Gateway. NAT allows outbound connections; it does not make the private task directly accessible from the internet.

NAT Gateways have ongoing charges. Production designs often use one per Availability Zone for resilience, which costs more. A learning project can consider temporary public-subnet tasks with tightly restricted inbound access, or VPC endpoints. Endpoint cost depends on endpoint type and service.

Cost reminder A NAT Gateway, ALB, and running Fargate tasks can continue to incur charges while idle. Check current pricing for your region before creating resources.

4. Security groups and IAM roles

Security groups are stateful virtual firewalls attached to AWS resources. A simple traffic pattern is:

- ALB security group: allow inbound HTTP port 80 from the intended users. For a temporary public demo this may be 0.0.0.0/0. For HTTPS, configure a certificate and HTTPS listener.
- ECS task security group: allow inbound TCP port 5000 only from the ALB security group.
- DynamoDB is a managed service, not an instance inside the VPC. Do not create public inbound rules for it.

AWS services should use IAM roles rather than access keys in code. The ECS task role is for application API calls. The task execution role lets ECS pull the image and send logs. Lambda uses its execution role for data services.

Role	Purpose
ECS task role	sqs:SendMessage on the exact registration queue.
ECS task execution role	Pull the image from ECR and write container logs to CloudWatch Logs.
Lambda execution role	Consume SQS messages; dynamodb:PutItem on the project table; optional sns:Publish on the topic.

Limit policies to the resource ARNs each role needs. Avoid broad administrator permissions for running applications.

5. ECR and the Docker image.

1. In ECR, create a repository such as smart-college-app.
2. From the project folder, build the image with the supplied Dockerfile. It installs requirements.txt, copies app.py and templates, exposes port 5000, and starts Flask.
3. Use the ECR push commands displayed for your repository. They include Docker login, image tagging, and push commands using your account and region.
4. Copy the full image URI. You will use it in the ECS task definition.

Before publishing Replace account IDs, ARNs, and queue URLs with placeholders. Never commit AWS access keys or real student data.

6. ECS and the web application

Amazon ECS runs containers. A task definition is the recipe for a container. An ECS service keeps the requested number of tasks running and can connect them to a load balancer.

Create the task definition

- Choose Fargate if using serverless container capacity. Select CPU and memory suitable for a small demo.
- Set the container image to the ECR URI and the container port to 5000.
- Set AWS_REGION and SQS_QUEUE_URL. Set FLASK_SECRET_KEY securely for a public deployment.
- Select the ECS task role and execution role. Configure CloudWatch logging.

Create an ECS service and ALB

- Create a target group using IP targets on port 5000. Set its health check path to /health, implemented by app.py.
- Create or select an internet-facing ALB in at least two public subnets. Attach the ALB security group and an HTTP listener for the demo.
- Create the ECS service with the task definition, subnets, and ECS task security group. Attach it to the target group.
- Wait for the task to become healthy, then open the ALB DNS name in a browser. Use HTTPS and a domain for a public deployment.

7. SQS queue and Lambda processing

Create the SQS queue

1. Open SQS and create a queue named smart-sqs. A Standard queue suits a basic demo unless strict ordering or FIFO behavior is required.
2. Keep the queue URL for the ECS environment variable and the queue ARN for IAM policy scoping.
3. Set message retention and visibility timeout deliberately. The visibility timeout should allow the Lambda invocation to finish before the message becomes available for retry.

Deploy the Lambda handler

1. Create a Lambda function in the same region as the SQS queue and DynamoDB table. Select a supported Python runtime.
2. Upload the handler source or paste it in the console. If the file is lambda_function.py and the function is lambda_handler, set the handler to lambda_function.lambda_handler.
3. Set DYNAMODB_TABLE and SNS_TOPIC_ARN as environment variables. Use the same region as the resources.
4. Add the SQS queue as a trigger and ensure the Lambda role can consume its messages.
5. If the handler returns batchItemFailures, enable **Report batch item failures** in the event source mapping. The project's simpler lambda_function.py does not use this partial-failure response.

Handler schema The supplied project handler stores the student name in the DynamoDB attribute name. The separately pasted handler uses student_name and adds created_at. Choose one version and keep its field names consistent with any code that reads the table.

8. DynamoDB and SNS

Create the DynamoDB table

1. Open DynamoDB, choose Tables, then Create table.
2. Use a table name such as dynamo-smart. Set the partition key to student_id with type String.
3. Choose a billing mode suited to demo traffic and check current regional pricing.
4. Give Lambda dynamodb:PutItem on this table. Test the handler and confirm the item appears in the table.

Configure SNS only if notifications are needed

1. Create an SNS topic and set its ARN in the Lambda SNS_TOPIC_ARN environment variable.
2. Create an email subscription if desired, then confirm it using the message SNS sends.
3. Give Lambda sns:Publish on the topic. Check logs and subscription status if no email arrives.

9. End-to-end test

1. Use dummy student data. Open the ALB DNS address and submit the form.
2. Confirm the form shows success. This means the app submitted the message to SQS, not that all processing has finished.
3. Check the Lambda CloudWatch log group for a successful invocation or an error.
4. Check DynamoDB for the new item and expected field names.
5. If SNS is enabled, check the email subscription and delivery status.
6. For failures, inspect the SQS message and Lambda error log. Check IAM permissions, region, table key, and JSON field names.

10. Cost control and cleanup

Common ongoing costs include NAT Gateways, ALBs, running Fargate tasks, public IPv4 addresses, data transfer, stored logs, and ECR images. SQS, Lambda, and DynamoDB charges depend on usage and configuration. Review current regional pricing and the AWS Billing console.

When the demo is finished

1. Set the ECS service desired count to zero or delete the service. Delete the cluster if it is empty and no longer needed.
2. Delete the ALB and target group.
3. Delete unused ECR images and repository.
4. Disable the Lambda SQS trigger, then delete the function if it is no longer needed.
5. Delete SQS, SNS, and DynamoDB resources if their data is disposable. Export data first if you need to keep it.
6. Delete NAT Gateways and release their Elastic IPs. Remove VPC endpoints created for the demo.
7. After attached resources are gone, remove custom subnets, route tables, security groups, and the custom VPC. Do not casually delete a shared default VPC.
8. Check CloudWatch log groups, ECR storage, public IP addresses, backups, and Billing for leftovers.

11. Future improvements

The current version is an educational demo. For a production-ready version, I would consider these improvements to strengthen security, reliability, and operations.

Planned improvement	Why it helps
HTTPS and a custom domain	Use a domain name, an AWS Certificate Manager certificate, and an HTTPS listener on the ALB to encrypt browser traffic. Configure HTTP to redirect to HTTPS.
Reliable message processing	Add an SQS dead-letter queue and retry policy. Make Lambda processing idempotent so retries do not create unintended duplicate work.
Monitoring and alerts	Use CloudWatch logs, metrics, and alarms for Lambda failures, queue backlog, and unhealthy ECS tasks.
Infrastructure as code and CI/CD	Use CloudFormation, CDK, or Terraform to recreate resources consistently. Automate image build and deployment.
Backups and data protection	Set backup and recovery options, encryption, and a retention policy for student information.
RDS when relational data is required	Consider Amazon RDS for MySQL if requirements need relational tables, SQL joins, or transactions. The current project uses DynamoDB for key-based records. RDS should solve a specific data need because it adds cost and operational work.